Concurrent Railway Reservation System

A working MVP: one seat, two simultaneous requests, exactly one winner.

Guru Pranesh M 192512439
Kamalanathan R.K 192512158
Revanth B 192524289

Hackathon Challenge Track

The Problem

Multiple booking counters and online users can request the same seat at nearly the same instant.

1

Dynamic seat allocation

seats sized per selected train, not hard-coded

2

Structs + pointers

passenger & train data modeled natively in C

3

Booking / cancel / refund / search

four core operations

4

Pthreads

simulate many concurrent users

5

Mutexes

prevent double-booking under a race

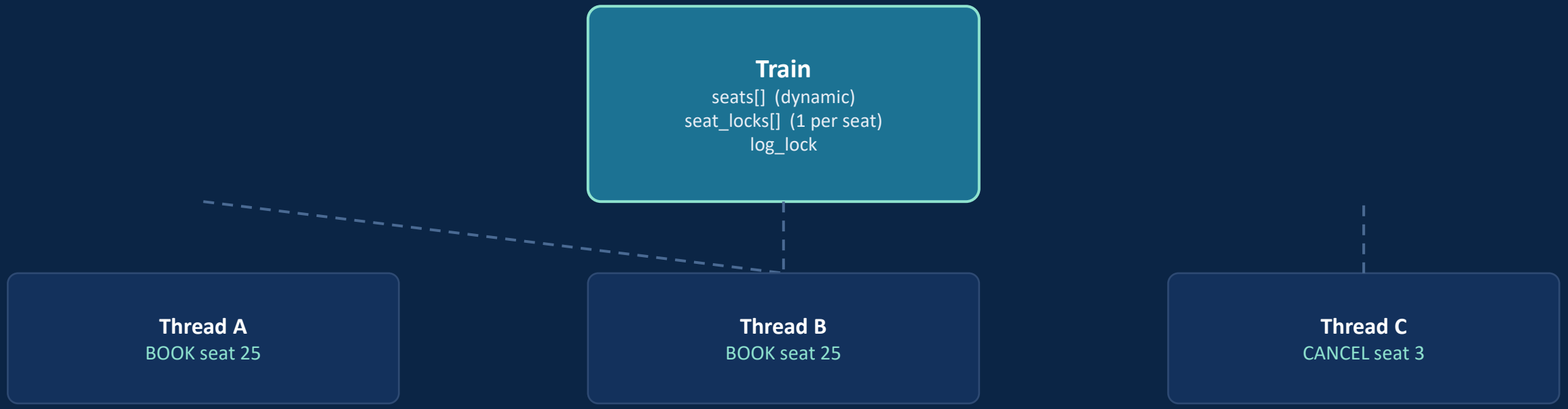
6

File-based transaction log

auditable record of every outcome

Architecture

One shared seat table. One mutex per seat. Many threads.



Both A and B lock the SAME seat's mutex — only one proceeds; the other fails fast and is refunded.

Guarantee: exactly one BOOK succeeds per seat, regardless of thread timing — enforced by pthread_mutex_lock/unlock around the check-then-set, not by luck.

Live Demo: The Race

Alice and Bob both request seat 25 in the same batch of concurrent threads.

```
$ ./reservation 12345 30 requests.txt

[thread 0] BOOK  Alice   -> SUCCESS, seat 25
[thread 1] BOOK  Bob     -> FAILED (seat 25 unavailable) -> REFUND ISSUED
[thread 5] CANCEL Alice  -> SUCCESS, seat 25 refunded
[thread 6] BOOK  Frank   -> SUCCESS, seat 25

Seat 25: BOOKED by Frank
```

Result: 1 success, 1 fail, every run — repeated 3x with identical outcome.

transactions.log

```
BOOK SUCCESS name=Alice seat=25
BOOK FAIL name=Bob seat=25 reason=ALREADY_BOOKED_by=Alice
```

Why it's not a fluke

Per-seat mutex makes check-then-set atomic — no interleaving window exists for both threads to see the seat as free.

The Critical Section

book_seat() — the ~10 lines that make the guarantee true

```
pthread_mutex_lock(&t->seat_locks[idx]);      // (1) claim this seat's lock

if (t->seats[idx].is_booked) {                //     the losing thread lands
    pthread_mutex_unlock(&t->seat_locks[idx]); //     here: seat already taken
    log_transaction(t, "BOOK FAIL ... ALREADY_BOOKED_by=%s", ...);
    return -1;                                //     -> caller reports refund
}

t->seats[idx].is_booked = 1;                  // (2) the winning thread claims
strncpy(t->seats[idx].passenger_name, name, ...);   // it, still holding the lock
pthread_mutex_unlock(&t->seat_locks[idx]);
log_transaction(t, "BOOK SUCCESS name=%s seat=%d", name, seat_no);
return seat_no;
```

What's Included & Next Steps

Done today

- ✓ Dynamic seat array sized from CLI seat_count
- ✓ Struct-based Train / Seat / Request models
- ✓ book / cancel / search functions, refund as a response
- ✓ One pthread per simulated request
- ✓ Per-seat mutex — race-safe, verified 3x
- ✓ Transaction log written to file, mutex-protected
- ✓ Bonus: TCP socket server, tested with a live client

With more time

- → Real ledger for refunds, not just a response message
- → Multi-train support (array of Train* keyed by train no.)
- → Persist seat state across restarts, replay the log
- → Multiple real client processes over sockets, not just threads
- → Waitlist / priority queue when a train is full